Combinaison de classifieurs et méthodes d'ensemble

Table des matières

1 Combinaison de classifieurs **2**
1.1 Définition et phases de la combinaison de classifieurs 2
1.1.1 Nature des sorties et niveaux d'information 3
1.2 Motivations théoriques et justifications statistiques 3
1.2.1 Théorème du jury de Condorcet 3
1.2.2 Décomposition biais–variance 4
1.2.3 Théorie unifiée de la diversité dans les ensembles 4
1.2.4 Les problèmes de Dietterich 5
1.3 Typologies des méthodes d'ensemble 6
1.3.1 Bagging (bootstrap aggregating) 6
1.3.2 Boosting 9
1.3.3 Stacking (stacked generalization) 9
1.3.4 Voting (vote majoritaire vs vote probabiliste) 10
1.4 Analyse comparative des approches 11
1.5 Aspects architecturaux des systèmes multi-classifieurs 11
1.5.1 Architectures de contrôle de flux 11
1.5.2 Systèmes de sélection dynamique (DS) 11
1.6 Avantages et limites de la combinaison 12
1.6.1 Avantages 12
1.6.2 Limites 12

Chapitre 1

Combinaison de classifieurs

Introduction

L'évolution de l'apprentissage automatique a franchi une étape décisive avec l'émergence des systèmes de classifieurs multiples (*Multiple Classifier Systems – MCS*), plus communément désignés sous le terme d'apprentissage d'ensemble [9, 27, 15]. Cette approche repose sur un paradigme fondamental : la combinaison stratégique de plusieurs modèles de prédiction pour pallier les insuffisances inhérentes à tout classifieur unique.

Historiquement, la recherche en intelligence artificielle s'est concentrée sur l'optimisation d'algorithmes individuels. Cependant, l'expérience a démontré qu'un modèle unique, aussi sophistiqué soit-il, est souvent limité par ses propres biais structurels et sa sensibilité au bruit des données. L'apprentissage d'ensemble propose de substituer au modèle solitaire un comité de décideurs dont la sagesse collective surpasse les capacités individuelles.

Cette section propose une exploration de la combinaison de classifieurs, structurée autour des fondements théoriques, des typologies méthodologiques et des considérations architecturales qui définissent ce domaine. L'analyse s'attache à montrer comment la diversité des modèles et la rigueur de leur agrégation permettent d'atteindre des niveaux de performance et de robustesse inaccessibles aux approches classiques.

1.1 Définition et phases de la combinaison de classifieurs

La **combinaison de classifieurs** est définie comme le processus d'intégration des sorties de plusieurs modèles d'apprentissage afin de générer une décision globale plus fiable et précise. Un système multi-classifieurs est structurellement composé d'un ensemble d'apprenants de base, souvent qualifiés d'*apprenants faibles* (*weak learners*) lorsqu'ils sont pris isolément, et d'un mécanisme de fusion qui orchestre leurs réponses.

Le cycle de vie d'un tel système se décompose généralement en trois phases critiques :

1. **Génération de l'ensemble** : cette phase initiale consiste à produire un pool de modèles de base. L'objectif est de créer des modèles qui ne sont pas seulement performants, mais surtout complémentaires. La complémentarité est assurée par l'introduction de diversité, que ce soit par l'utilisation de différents sous-ensembles de données, de paramètres variés pour un même algorithme, ou de familles d'algorithmes distinctes.
2. **Sélection ou élagage (*pruning*)** : une fois le pool généré, il peut être nécessaire de réduire la taille de l'ensemble. L'élagage vise à éliminer les modèles redondants ou ceux dont la performance dégrade la décision collective. Cette étape permet non seulement d'améliorer la précision globale mais aussi de réduire les coûts computationnels lors de l'inférence.

3. **Combinaison ou fusion** : la phase finale consiste à fusionner les prédictions des modèles sélectionnés pour obtenir la réponse ultime. Le choix de la fonction de combinaison dépend crucialement du type d'information fournie par les classifieurs de base (niveau d'information en sortie).

1.1.1 Nature des sorties et niveaux d'information

L'efficacité de la combinaison dépend de la richesse des informations transmises par les modèles individuels. On distingue classiquement trois niveaux de sortie :

— **Niveau « Classe » (hard output)** : le classifieur fournit uniquement l'étiquette de la classe prédite. C'est l'information la plus élémentaire, où chaque modèle agit comme un votant binaire.

— **Niveau « Rang »** : le classifieur produit une liste ordonnée des classes possibles, de la plus probable à la moins probable. Ce format est particulièrement utile lorsque la classe réelle ne figure pas au premier rang mais reste parmi les premières propositions.

— **Niveau « Mesure » (soft output)** : c'est le niveau le plus riche. Le classifieur associe à chaque classe une valeur numérique, telle qu'une probabilité ou un score de confiance. Cette mesure quantifie l'incertitude du modèle et permet des fusions pondérées beaucoup plus fines.

Le tableau suivant présente une comparaison structurelle de ces types d'informations en sortie.

Table 1.1 – Comparaison des niveaux d'information en sortie des classifieurs

Type de sortie	Information contenue	Complexité de fusion	Cas d'utilisation typique
Classe	Étiquette discrète uniquement	Faible	Vote majoritaire simple
Rang	Préférences relatives ordonnées	Moyenne	Méthode de Borda, systèmes de recommandation
Mesure	Probabilités, scores de confiance	Élevée	Moyenne pondérée, stacking, modèles bayésiens

1.2 Motivations théoriques et justifications statistiques

La supériorité des systèmes multi-classifieurs n'est pas une simple observation empirique ; elle s'appuie sur des piliers théoriques robustes, notamment le théorème du jury de Condorcet et la décomposition biais–variance [24, 1, 10].

1.2.1 Théorème du jury de Condorcet

Formulé à l'origine dans le contexte des sciences sociales, ce théorème constitue une base théorique fondamentale pour l'apprentissage d'ensemble [28, 5]. Il postule que si un groupe d'individus doit prendre une décision binaire (par exemple, classer un objet comme A ou B) et que chaque individu a une probabilité p supérieure à 0,5 de choisir la bonne réponse, alors la probabilité que la décision de la majorité soit correcte augmente avec le nombre d'individus.

Mathématiquement, si L est le nombre de classifieurs indépendants et p leur précision individuelle, la probabilité P que l'ensemble prenne la bonne décision par vote majoritaire tend vers 1 lorsque L tend vers l'infini. À l'inverse, si $p < 0{,}5$, l'ajout de classifieurs réduit la performance globale. Cette démonstration justifie l'utilisation d'« apprenants faibles » pour construire des systèmes puissants, à condition qu'ils soient plus performants qu'un tirage aléatoire et qu'ils soient suffisamment indépendants.

1.2.2 Décomposition biais–variance

D'un point de vue statistique, l'erreur de généralisation d'un modèle d'apprentissage peut être décomposée en trois composantes : le biais au carré, la variance et l'erreur irréductible (bruit) [26, 12]. La formule de l'erreur quadratique moyenne (MSE) est :

$$\mathbb{E}\big[(y - \hat{f}(x))^2\big] = \mathrm{Bias}\big[\hat{f}(x)\big]^2 + \mathrm{Var}\big[\hat{f}(x)\big] + \sigma^2, \tag{1.1}$$

où σ^2 représente le bruit inhérent aux données.

— Le **biais** mesure l'écart entre la prédiction moyenne du modèle et la valeur réelle. Un biais élevé est le signe d'un modèle trop simpliste qui ne capture pas la complexité des données (sous-apprentissage, *underfitting*).

— La **variance** mesure la dispersion des prédictions pour différents jeux de données d'apprentissage. Une variance élevée indique que le modèle est trop sensible aux fluctuations et au bruit des données (sur-apprentissage, *overfitting*).

L'apprentissage d'ensemble permet de naviguer dans ce compromis biais–variance. Les méthodes de type bagging agissent principalement en réduisant la variance par la moyenne de modèles complexes, tandis que les méthodes de boosting se concentrent sur la réduction du biais en agrégeant itérativement des modèles simples pour corriger leurs erreurs systématiques.

1.2.3 Théorie unifiée de la diversité dans les ensembles

Une contribution théorique importante aux méthodes d'ensemble consiste à faire apparaître explicitement la diversité comme composante de la décomposition de la perte. Pour une vaste famille de fonctions de perte (notamment les divergences de Bregman, qui incluent l'erreur quadratique moyenne et l'entropie croisée), on peut montrer que la perte de l'ensemble s'exprime comme la moyenne des pertes individuelles diminuée d'un terme de diversité [20].

Dans le cas particulier de l'erreur quadratique moyenne, cette idée est capturée par l'identité de Krogh et Vedelsby [19]. Pour un ensemble de M modèles dont les prédictions sont $q_1(x), \ldots, q_M(x)$ pour une entrée x et une cible y, on pose

$$\bar{q}(x) = \frac{1}{M}\sum_{i=1}^{M} q_i(x).$$

L'identité s'écrit alors :

$$\big(\bar{q}(x) - y\big)^2 = \frac{1}{M}\sum_{i=1}^{M}\big(q_i(x) - y\big)^2 - \frac{1}{M}\sum_{i=1}^{M}\big(q_i(x) - \bar{q}(x)\big)^2. \tag{1.2}$$

Le membre de gauche est l'erreur quadratique de l'ensemble, le premier terme à droite la moyenne des erreurs quadratiques individuelles, et le second terme (souvent appelé « ambiguïté » ou « diversité ») mesure le degré de désaccord entre les prédictions des modèles. L'identité montre que, toutes choses égales par ailleurs, l'erreur de l'ensemble diminue lorsque la diversité augmente.

Démonstration de l'identité de Krogh et Vedelsby. Pour simplifier les notations, on note $\bar{q} = \bar{q}(x)$ et $q_i = q_i(x)$.

(i) Développement de l'erreur de l'ensemble.

$$(\bar{q} - y)^2 = \bar{q}^2 - 2\bar{q}y + y^2.$$

(ii) Développement de la moyenne des erreurs individuelles.

$$\frac{1}{M}\sum_{i=1}^{M}(q_i - y)^2 = \frac{1}{M}\sum_{i=1}^{M}(q_i^2 - 2q_i y + y^2) \tag{1.3}$$

$$= \frac{1}{M}\sum_{i=1}^{M} q_i^2 - 2y\left(\frac{1}{M}\sum_{i=1}^{M} q_i\right) + y^2 \tag{1.4}$$

$$= \frac{1}{M}\sum_{i=1}^{M} q_i^2 - 2y\bar{q} + y^2. \tag{1.5}$$

(iii) Développement du terme de diversité.

$$\frac{1}{M}\sum_{i=1}^{M}(q_i - \bar{q})^2 = \frac{1}{M}\sum_{i=1}^{M}(q_i^2 - 2q_i\bar{q} + \bar{q}^2) \tag{1.6}$$

$$= \frac{1}{M}\sum_{i=1}^{M} q_i^2 - 2\bar{q}\left(\frac{1}{M}\sum_{i=1}^{M} q_i\right) + \bar{q}^2 \tag{1.7}$$

$$= \frac{1}{M}\sum_{i=1}^{M} q_i^2 - 2\bar{q}^2 + \bar{q}^2 \tag{1.8}$$

$$= \frac{1}{M}\sum_{i=1}^{M} q_i^2 - \bar{q}^2. \tag{1.9}$$

(iv) Soustraction des deux termes.

$$\frac{1}{M}\sum_{i=1}^{M}(q_i - y)^2 - \frac{1}{M}\sum_{i=1}^{M}(q_i - \bar{q})^2 = \left(\frac{1}{M}\sum_{i=1}^{M} q_i^2 - 2y\bar{q} + y^2\right) - \left(\frac{1}{M}\sum_{i=1}^{M} q_i^2 - \bar{q}^2\right) \tag{1.10}$$

$$= -2y\bar{q} + y^2 + \bar{q}^2 \tag{1.11}$$

$$= (\bar{q} - y)^2. \tag{1.12}$$

On retrouve bien l'identité annoncée. Comme le terme de diversité est une somme de carrés, il est toujours positif ou nul. L'erreur de l'ensemble est donc toujours inférieure ou égale à la moyenne des erreurs individuelles, avec égalité lorsque tous les modèles prédisent la même valeur (diversité nulle).

Cette identité justifie l'importance de promouvoir la diversité dans les ensembles (via le bagging, le boosting, la randomisation des architectures, etc.) et ouvre sur des généralisations à d'autres pertes au moyen des divergences de Bregman [20].

1.2.4 Les problèmes de Dietterich

Thomas Dietterich a identifié trois raisons fondamentales expliquant pourquoi un ensemble peut surpasser un modèle unique :

1. **Le problème statistique** : lorsque le volume de données est faible par rapport à la taille de l'espace des hypothèses, l'algorithme peut trouver plusieurs modèles qui s'ajustent parfaitement aux données d'entraînement. En les combinant, l'ensemble réduit le risque de choisir par erreur un modèle qui généralise mal.
2. **Le problème computationnel** : de nombreux algorithmes (comme les réseaux de neurones) utilisent des heuristiques de recherche locale qui peuvent rester bloquées dans des optima locaux. Un ensemble effectuant plusieurs recherches à partir de points de départ différents a plus de chances de trouver une meilleure approximation globale.

3. **Le problème de représentation** : parfois, la véritable fonction cible ne peut être représentée par aucun des modèles de l'espace d'hypothèses choisi. La combinaison de plusieurs modèles permet d'étendre cet espace et d'approcher des fonctions plus complexes.

1.3 Typologies des méthodes d'ensemble

Les méthodes d'ensemble se divisent principalement en approches parallèles (bagging), séquentielles (boosting) et hiérarchiques (stacking) [3, 29, 22].

1.3.1 Bagging (bootstrap aggregating)

Le bagging (pour *bootstrap aggregating*) vise à réduire la variance en entraînant plusieurs modèles indépendants en parallèle sur des échantillons de données légèrement différents. Introduit par Breiman [2] et largement diffusé ensuite [16, 13, 1], il est particulièrement efficace pour des apprenants de base instables comme les arbres de décision profonds.

Définition. À partir d'un ensemble d'apprentissage $D = \{(x_j, y_j)\}_{j=1}^{n}$, le bagging construit m échantillons bootstrap D_i par tirage aléatoire avec remise, chacun de taille n. Sur chaque D_i, on entraîne un modèle de base $\hat{f}_i$ (*base learner*). À la prédiction, les sorties des m modèles sont agrégées :

— en régression, par moyenne

$$\hat{f}(x) = \frac{1}{m} \sum_{i=1}^{m} \hat{f}_i(x);$$

— en classification, par vote majoritaire

$$\hat{y}(x) = \arg\max_{k} \sum_{i=1}^{m} \mathbb{I}\big(\hat{y}_i(x) = k\big).$$

La construction des m modèles étant indépendante, l'entraînement est naturellement parallélisable et s'adapte bien aux architectures multi-cœurs ou distribuées [3, 16].

Fonctionnement détaillé. On peut résumer la procédure de bagging en quatre étapes [2, 21] :

1. **Choix de l'apprenant de base et du nombre de modèles** : on fixe un modèle de base (par exemple, un arbre de décision) et un nombre de réplications m (souvent entre 50 et 500).
2. **Génération des échantillons bootstrap** : pour chaque $i \in \{1, \ldots, m\}$, on tire n exemples de D avec remise pour former D_i. Chaque observation a une probabilité $1/n$ d'être choisie à chaque tirage ; au total, environ 63,2 % des exemples distincts apparaissent dans D_i, les autres constituant les données *out-of-bag*.
3. **Entraînement des modèles** : on entraîne indépendamment un modèle $\hat{f}_i$ sur chaque D_i. Cette étape est triviale à paralléliser.
4. **Agrégation des prédictions** : pour une nouvelle entrée x, on calcule les prédictions des m modèles et on les agrège par moyenne (régression) ou vote (classification), comme indiqué ci-dessus.

Un pseudocode simplifié est le suivant :

```
fonction Bagging(D, m, base_learner):
    modèles = []
    pour i de 1 à m:
        D_i = bootstrap_sample(D)      // tirage avec remise
```

```
        f_i = base_learner.fit(D_i)
        modèles.ajouter(f_i)
    retour modèles

fonction Predire(modèles, x):
    prédictions = [f_i.predict(x) pour f_i dans modèles]
    si régression:      retour moyenne(prédictions)
    si classification:  retour mode(prédictions)
```

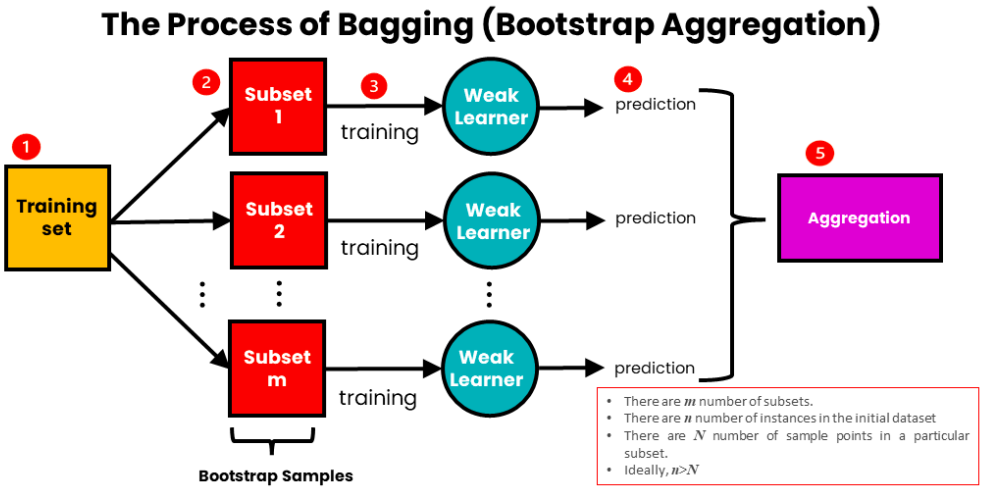

FIGURE 1.1 – Schéma pédagogique du bagging : du jeu de données original aux échantillons bootstrap, puis aux modèles de base et à l'agrégation.

Rôle des données out-of-bag (OOB). Lors de la génération de D_i, chaque exemple de D a une probabilité $(1-1/n)^n \simeq 1/e$ de ne jamais être sélectionné ; environ 36,8 % des observations restent donc en dehors de chaque bootstrap et sont dites *out-of-bag* pour le modèle $\hat{f}_i$ considéré.

Pour une observation (x_j, y_j), on note

$$S_j = \{\, i \in \{1, \dots, m\} \mid (x_j, y_j) \notin D_i \,\}$$

l'ensemble des indices de modèles qui n'ont pas vu ce point à l'entraînement. On peut alors définir une prédiction OOB par

$$\tilde{f}_j(x_j) = \frac{1}{|S_j|} \sum_{i \in S_j} \hat{f}_i(x_j)$$

en régression, ou par vote majoritaire restreint aux $i \in S_j$ en classification. Une estimation de l'erreur OOB est donnée par

$$\varepsilon_{\text{OOB}} = \frac{1}{n} \sum_{j=1}^{n} \ell\big(\tilde{f}_j(x_j), y_j\big),$$

où ℓ est la fonction de perte (MSE, entropie croisée, etc.). Cette quantité fournit une approximation fiable de l'erreur de généralisation, sans avoir besoin de réserver un jeu de validation distinct [2, 21, 16].

Réduction de la variance. Le principal intérêt du bagging est de réduire la variance des prédictions. Dans un scénario idéal où les modèles $\hat{f}_i$ seraient indépendants et de variance v, la variance de la moyenne

$$\hat{f}(x) = \frac{1}{m} \sum_{i=1}^{m} \hat{f}_i(x)$$

serait v/m. En pratique, les modèles ne sont pas indépendants ; si l'on note ρ la corrélation moyenne entre deux prédicteurs, on obtient approximativement [5] :

$$\mathrm{Var}\big[\hat{f}(x)\big] \approx \rho v + \frac{1-\rho}{m}\, v.$$

Quand m augmente, le second terme décroît comme $1/m$ et la variance tend vers ρv. Le bagging est donc particulièrement pertinent lorsque les modèles de base sont peu biaisés mais très variables (comme les arbres profonds) : il stabilise leurs prédictions sans augmenter significativement le biais.

Exemples d'algorithmes baggés. *Forêt aléatoire.* La forêt aléatoire (*Random Forest*) est l'extension la plus célèbre du bagging. En plus du bootstrapping, elle introduit une randomisation des caractéristiques (*features*) : à chaque division d'un nœud de l'arbre, seul un sous-ensemble aléatoire de variables est considéré. Cela décorrèle les arbres et renforce encore la réduction de variance [2, 16].

Un aperçu schématique de l'algorithme Random Forest est donné par la figure 1.2.

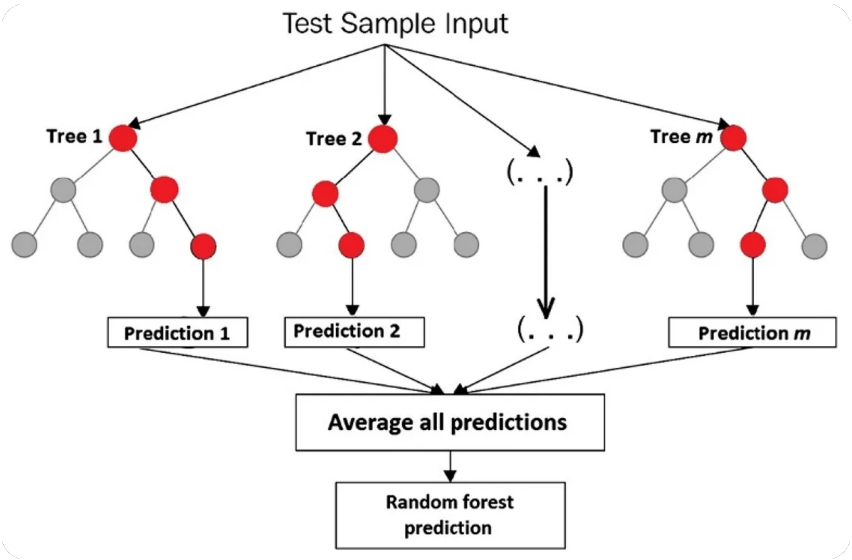

FIGURE 1.2 – Aperçu de l'algorithme Random Forest.

Exemple : bagging avec k plus proches voisins. Le bagging ne se limite pas aux arbres. Par exemple, appliqué au classifieur des k plus proches voisins (k-NN), il permet de stabiliser un modèle naturellement très sensible aux fluctuations locales des données. On génère m jeux bootstrap D_i, chacun servant de base à un modèle k-NN (qui consiste essentiellement à mémoriser D_i). Pour une nouvelle observation x, on agrège les m prédictions k-NN par vote ou moyenne des probabilités estimées [10, 3].

Dans les bibliothèques modernes comme scikit-learn, ces variantes sont disponibles via la classe `BaggingClassifier` en choisissant simplement l'estimateur de base (arbre, k-NN, etc.) et en activant le calcul de la « OOB score » pour obtenir automatiquement une estimation interne de l'erreur de généralisation.

1.3.2 Boosting

Le boosting est une approche séquentielle où chaque modèle tente de corriger les erreurs de son prédécesseur, réduisant ainsi principalement le biais [3, 13].

Mécanisme d'AdaBoost (Adaptive Boosting).

1. **Initialisation** : on attribue un poids identique $w_i = 1/n$ à chaque exemple.
2. **Itération** : on entraîne un modèle faible (souvent un « stump », arbre de profondeur 1) et on calcule son erreur pondérée ϵ.
3. **Mise à jour des poids** : les exemples mal classés voient leur poids augmenter, forçant le modèle suivant à se concentrer sur eux. Les exemples bien classés voient leur poids diminuer.
4. **Combinaison** : la décision finale est une somme pondérée des modèles par leur coefficient de confiance α_t.

Gradient boosting. Dans le gradient boosting (XGBoost, LightGBM, etc.), plutôt que de repondérer les points, on entraîne chaque nouveau modèle pour prédire le résidu (l'erreur) du modèle précédent. On descend ainsi le gradient de la fonction de perte pour minimiser l'erreur globale étape par étape.

Le texte décrit un « Schéma pédagogique du boosting » sous forme de chaîne séquentielle (« Modèle 1 », mise à jour des poids / calcul des résidus, « Modèle 2 », etc.) jusqu'à la combinaison finale. Ce contenu peut être illustré par une figure.

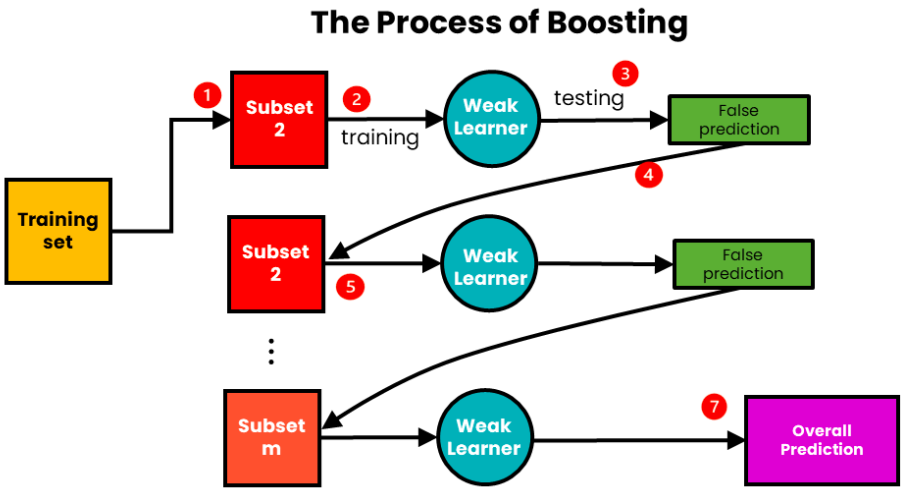

FIGURE 1.3 – Schéma pédagogique du boosting : chaîne de modèles où chaque étape corrige les erreurs de la précédente.

1.3.3 Stacking (stacked generalization)

Le stacking utilise un modèle d'apprentissage pour apprendre comment combiner au mieux les prédictions de plusieurs modèles de base [29, 14].

Architecture en couches.

— **Niveau 0 (base learners)** : des modèles hétérogènes (par exemple SVM, forêt aléatoire, k-NN) sont entraînés sur les données.

— **Méta-caractéristiques** : les prédictions de ces modèles servent de nouvelles variables d'entrée.
— **Niveau 1 (méta-modèle)** : un dernier classifieur (souvent une régression logistique) apprend à pondérer ces prédictions.

Rôle de la validation croisée. Pour éviter le surapprentissage (le méta-modèle pourrait favoriser des modèles qui ont simplement mémorisé le jeu d'entraînement), on utilise des prédictions *out-of-fold*. Chaque modèle de base prédit des données qu'il n'a pas vues durant son entraînement partiel, ce qui garantit que le méta-modèle apprend sur des performances réelles de généralisation.

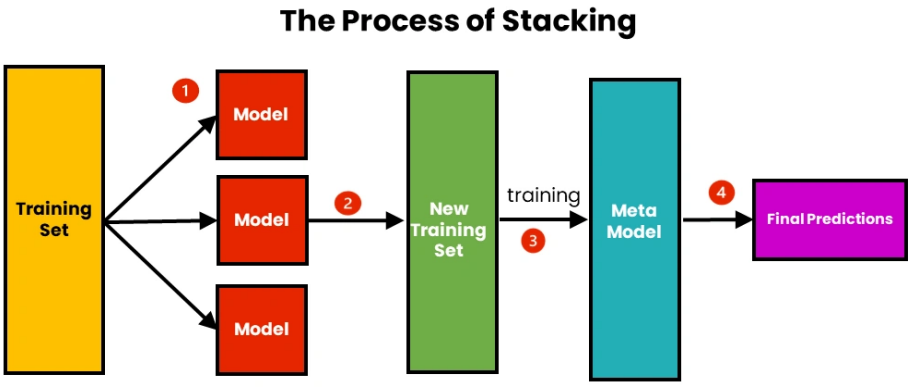

FIGURE 1.4 – Schéma pédagogique du stacking : architecture en couches avec base learners et méta-modèle.

1.3.4 Voting (vote majoritaire vs vote probabiliste)

Le vote est la méthode de fusion la plus simple pour les modèles parallèles [4, 11].

Hard voting (vote majoritaire). Chaque modèle donne une étiquette de classe. On choisit la classe qui a reçu le plus de suffrages. Par exemple :
— Modèle A → « chien »
— Modèle B → « chat »
— Modèle C → « chien »
La décision finale est alors « chien » (deux voix contre une).

Soft voting (vote probabiliste). On calcule la moyenne des probabilités prédites pour chaque classe. Cette méthode est plus précise car elle accorde plus d'importance aux modèles « sûrs d'eux ».

Considérons par exemple deux classes (chat, chien) et trois modèles :
— Modèle 1 : probabilités $[P(\text{chien}) = 0{,}4,\ P(\text{chat}) = 0{,}6] \Rightarrow$ prédit « chat »
— Modèle 2 : probabilités $[0{,}4,\ 0{,}6] \Rightarrow$ prédit « chat »
— Modèle 3 : probabilités $[0{,}9,\ 0{,}1] \Rightarrow$ prédit « chien »
En hard voting, la classe « chat » (deux voix) l'emporte. En soft voting, on calcule :

$$\overline{P}(\text{chien}) = \frac{0{,}4 + 0{,}4 + 0{,}9}{3} \approx 0{,}56, \qquad \overline{P}(\text{chat}) = \frac{0{,}6 + 0{,}6 + 0{,}1}{3} \approx 0{,}43.$$

La décision finale est alors « chien » : la forte conviction du modèle 3 compense l'incertitude des deux autres.

1.4 Analyse comparative des approches

Le choix d'une méthode d'ensemble dépend des contraintes du projet (volume de données, temps de réponse, précision requise, robustesse au bruit, coût de calcul, etc.).

Le tableau suivant résume quelques caractéristiques essentielles des trois grandes familles :

TABLE 1.2 – Comparaison synthétique bagging / boosting / stacking

Critère	Bagging	Boosting	Stacking
Objectif principal	Réduire la variance	Réduire le biais	Maximiser la précis
Type de modèles	Homogènes (arbres profonds)	Homogènes (arbres peu profonds)	Hétérogènes (divers
Entraînement	Parallèle (indépendant)	Séquentiel (dépendant)	Parallèle + méta-a
Sensibilité au bruit	Faible (robuste)	Élevée (risque de sur-ajuster)	Variable
Risque d'overfitting	Faible	Élevé (si non régularisé)	Moyen (si données
Coût de calcul	Efficace (parallélisable)	Lent (entraînement en série)	Très élevé

1.5 Aspects architecturaux des systèmes multi-classifieurs

L'architecture d'un système multi-classifieurs définit la manière dont les informations circulent entre les composants et la façon dont les ressources sont exploitées.

1.5.1 Architectures de contrôle de flux

On distingue trois organisations principales :

1. **Architecture parallèle** : tous les classifieurs opèrent indépendamment et simultanément. La fusion intervient à la fin. Cette structure est neutre (aucun classifieur n'influence les autres) et hautement parallélisable sur du matériel multi-cœurs.
2. **Architecture séquentielle (en série)** : les classifieurs sont organisés en cascade. Un premier classifieur rapide effectue une prédiction ; s'il échoue ou exprime une faible confiance, l'exemple est passé à un second modèle plus complexe. Cela permet d'économiser des ressources de calcul en ne sollicitant les modèles coûteux que pour les cas difficiles.
3. **Architecture hybride** : elle combine les deux approches. Par exemple, un premier étage parallèle peut générer un consensus, dont le résultat est validé par un classifieur de contrôle séquentiel.

Ces formes peuvent être illustrées par des figures dédiées aux architectures parallèle et séquentielle.

1.5.2 Systèmes de sélection dynamique (DS)

Une tendance majeure consiste à ne pas utiliser tous les classifieurs pour chaque exemple, mais à sélectionner « à la volée » le ou les modèles les plus compétents [7, 23, 18, 6, 8].

— **Logique de compétence locale** : on définit une « région de compétence » autour d'un exemple test (en utilisant par exemple ses plus proches voisins dans le jeu d'entraînement). On évalue ensuite quels modèles du pool ont été les plus performants dans cette zone spécifique.

— **DCS (dynamic classifier selection)** : on choisit l'unique meilleur modèle pour l'exemple donné.

— **DES (dynamic ensemble selection)** : on construit un sous-ensemble de modèles experts pour cette région particulière.

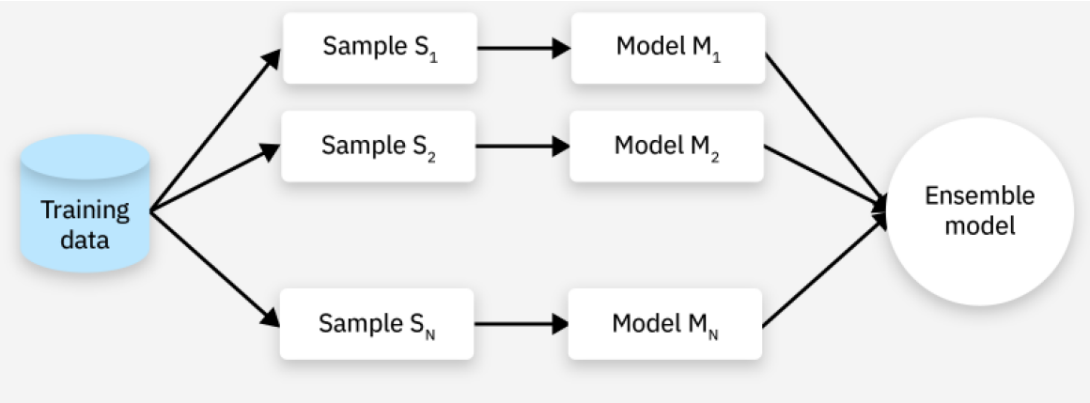

FIGURE 1.5 – Architecture parallèle pour un système multi-classifieurs.

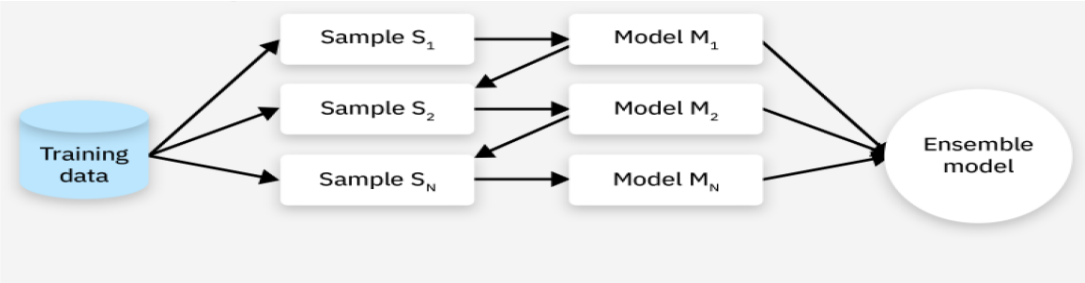

FIGURE 1.6 – Architecture séquentielle pour un système multi-classifieurs.

1.6 Avantages et limites de la combinaison

1.6.1 Avantages

1. **Précision supérieure** : les ensembles dépassent quasi systématiquement les modèles uniques en combinant des perspectives complémentaires.
2. **Généralisation et robustesse** : en lissant les prédictions, les systèmes MCS sont moins sensibles aux anomalies statistiques d'un seul jeu de données.
3. **Traitement des données complexes** : ils permettent de diviser un problème complexe en sous-tâches traitées par des experts dédiés (*divide and conquer*).

1.6.2 Limites

1. **Coût en ressources** : l'entraînement de dizaines ou de centaines de modèles consomme du temps CPU, de la mémoire et de l'énergie.
2. **Opacité (« black box »)** : il est extrêmement difficile d'expliquer pourquoi un ensemble de plusieurs centaines d'arbres de décision a pris une décision spécifique.
3. **Complexité de maintenance** : gérer un pipeline avec plusieurs modèles hétérogènes demande une infrastructure plus lourde qu'un modèle monolithique.

Conclusion

La combinaison de classifieurs représente une approche mature et extrêmement puissante de l'apprentissage automatique. En orchestrant judicieusement la diversité et en s'appuyant sur des fondements statistiques rigoureux, les systèmes multi-classifieurs permettent de transformer une collection de modèles imparfaits en un outil de décision d'une fiabilité exceptionnelle [17, 25, 10, 9].

Bibliographie

[1] Analytics Vidhya. Bagging, boosting and stacking : Ensemble learning in ml models. https://www.analyticsvidhya.com/blog/2023/01/ensemble-learning-methods-bagging-boosting-and-stacking/, 2023. Consulté le 23 février 2026.

[2] Leo Breiman. Bagging predictors. *Machine Learning*, 24(2) :123–140, 1996. Article fondateur du bagging.

[3] Jason Brownlee. Bagging vs boosting vs stacking : Which ensemble method wins in 2025 ? https://machinelearningmastery.com/bagging-vs-boosting-vs-stacking-which-ensemble-method-wins-in-2025/, 2025. Consulté le 23 février 2026.

[4] Codefinity. Learn voting : Hard and soft. https://codefinity.com/courses/v2/3172b303-2b09-492b-8bb9-5bbd1bdcef09/81f3a01b-d537-4478-bc23-3ea1895e4f1f/d66aa7bd-877e-4808-ad40-aa051eabbc4a, 2024. Consulté le 23 février 2026.

[5] William W. Cohen. Ensemble methods for machine learning. https://www.cs.cmu.edu/~wcohen/10-601/ensembles.pdf, 2010. Support de cours, Carnegie Mellon University. Consulté le 23 février 2026.

[6] R. M. O. Cruz, R. Sabourin, and G. D. C. Cavalcanti. Dynamic ensemble selection and instantaneous pruning for regression. In *ESANN 2014*, 2014. Consulté le 23 février 2026.

[7] R. M. O. Cruz, R. Sabourin, and G. D. C. Cavalcanti. Deslib : A dynamic ensemble selection library in python. *Journal of Machine Learning Research*, 21(8) :1–5, 2020. Consulté le 23 février 2026.

[8] DESlib project. Welcome to deslib documentation ! https://deslib.readthedocs.io/, 2020. Consulté le 23 février 2026.

[9] Luis Fred. Multiple classifier systems — a brief introduction. https://medium.com/luisfredgs/multiple-classifier-systems-a-brief-introduction-71238d9c794f, 2023. Consulté le 23 février 2026.

[10] GeeksforGeeks. A comprehensive guide to ensemble learning. https://www.geeksforgeeks.org/machine-learning/a-comprehensive-guide-to-ensemble-learning/, 2023. Consulté le 23 février 2026.

[11] GeeksforGeeks. Voting classifier. https://www.geeksforgeeks.org/machine-learning/voting-classifier/, 2023. Consulté le 23 février 2026.

[12] Vibhav Gogate. Bias/variance tradeoff and ensemble methods. https://personal.utdallas.edu/~vibhav.gogate/ml/2020f/lectures/EnsembleMethods.pdf, 2020. Support de cours, Université du Texas à Dallas. Consulté le 23 février 2026.

[13] Great Learning. Understanding the ensemble method bagging and boosting. https://www.mygreatlearning.com/blog/bagging-boosting/, 2023. Consulté le 23 février 2026.

[14] Henry Henry. Théorie – apprentissages d'ensemble : bagging, boosting et stacking. https://medium.com/@henryhenry_3807/th%C3%

A9orie-apprentissages-densemble-bagging-boosting-et-stacking-f3c8d2273edf, 2023. Consulté le 23 février 2026.

[15] Laurent Heutte. Combinaison de classifieurs. http://lheutte.free.fr/rdf/ArchiCombi.pdf, 2003. Consulté le 23 février 2026.

[16] IBM. What is bagging ? https://www.ibm.com/think/topics/bagging, 2024. Consulté le 23 février 2026.

[17] IBM. What is ensemble learning ? https://www.ibm.com/think/topics/ensemble-learning, 2024. Consulté le 23 février 2026.

[18] Albert Ko, Robert Sabourin, and Alceu de Souza Britto Jr. From dynamic classifier selection to dynamic ensemble selection. In *Proceedings of the 7th International Conference on Machine Learning and Applications*, 2007. Consulté le 23 février 2026.

[19] Anders Krogh and Jesper Vedelsby. Neural network ensembles, cross validation, and active learning. In *Advances in Neural Information Processing Systems 7 (NIPS 1994)*, pages 231–238, 1995. Article fondateur de l'identité de Krogh et Vedelsby sur l'ambiguïté des ensembles.

[20] Ludmila I. Kuncheva and Juan J. Rodríguez. A unified theory of diversity in ensemble learning. *Information Fusion*, 2023. Résumé disponible sur ResearchGate, consulté le 23 février 2026.

[21] G. Metzler. Machine learning td – méthodes ensemblistes. https://eric.univ-lyon2.fr/gmetzler/courses/contents/machinelearning/td4.pdf, 2022. Travaux dirigés, Université Lyon 2. Consulté le 23 février 2026.

[22] NS Academy. What is ensemble learning (bagging vs boosting vs stacking) ? https://medium.com/@sharetonschool/what-is-ensemble-learning-bagging-vs-boosting-vs-stacking-c3c8cc94a3d5, 2023. Consulté le 23 février 2026.

[23] scikit-learn-contrib. Deslib : A python library for dynamic classifier and ensemble selection. https://github.com/scikit-learn-contrib/DESlib, 2020. Consulté le 23 février 2026.

[24] Turing College. Ensemble learning : Key techniques explained. https://www.turingcollege.com/blog/ensemble-learning-2, 2024. Consulté le 23 février 2026.

[25] Ultralytics. Exploration de l'apprentissage d'ensemble : son rôle dans l'ia et la ml. https://www.ultralytics.com/fr/blog/exploring-ensemble-learning-and-its-role-in-ai-and-ml, 2024. Consulté le 23 février 2026.

[26] Wikipedia. Bias–variance tradeoff. https://en.wikipedia.org/wiki/Bias%E2%80%93variance_tradeoff, 2026. Consulté le 23 février 2026.

[27] Wikipedia. Ensemble learning. https://en.wikipedia.org/wiki/Ensemble_learning, 2026. Consulté le 23 février 2026.

[28] Wikipédia. Théorème du jury. https://fr.wikipedia.org/wiki/Th%C3%A9or%C3%A8me_du_jury, 2026. Consulté le 23 février 2026.

[29] Kaleemullah Younas. Bagging, boosting and stacking : Ensemble learning in ml models. https://medium.com/@kaleemullahyounas123/bagging-boosting-and-stacking-ensemble-learning-in-ml-models-8eb2f73afb07, 2023. Consulté le 23 février 2026.